

ARTIFICIAL INTELLIGENCE ASSESSMENT – 1

REPORT

Course Code: CSA1704 – Artificial Intelligence

Submitted By

Name: LAKSHITHA SREE K

Register Number: 192511336

Department: Computer Science and Engineering

Acknowledgement

I express my sincere gratitude to my faculty members and the Department of Computer Science and Engineering for providing me with the opportunity to complete this Artificial Intelligence Assessment successfully. I also thank my friends and family members for their continuous encouragement and support throughout the completion of this work. Finally, I thank everyone who directly or indirectly contributed to the successful completion of this assessment.

Table of Contents

1. Introduction
2. Objective
3. Question 1 – Water Jug Problem
4. Question 2 – Mars Rover Intelligent Agent
5. Question 3 – Eight Queens Problem
6. Question 4 – OLA Cab Booking Agent
7. Question 5 – Uniform Cost Search
8. Python Implementation
9. Program Outputs
10. Conclusion
11. References

Introduction

Artificial Intelligence (AI) is a branch of computer science that focuses on designing systems capable of performing tasks that normally require human intelligence. These tasks include learning, reasoning, planning, perception, decision making, problem solving, and natural language understanding. AI systems use various search techniques, intelligent agents, and optimization algorithms to solve complex real-world problems efficiently.

In this assessment, five important Artificial Intelligence problems have been studied and implemented. These problems demonstrate state-space search, intelligent agents, constraint satisfaction, goal-based reasoning, and graph search algorithms. Python programming language is used to implement the algorithms due to its simplicity and powerful libraries.

Objective

The objectives of this assessment are:

- To understand Artificial Intelligence concepts.
- To study state-space search problems.
- To implement intelligent agents.
- To solve constraint satisfaction problems.
- To implement graph search algorithms.

- To develop Python programs for AI applications.

Question 1 – Water Jug Problem

Problem Statement

The Water Jug Problem is a classic Artificial Intelligence state-space search problem. The objective is to obtain exactly 2 gallons of water in a 4-gallon jug using only two unmarked jugs of capacities 4 gallons and 3 gallons. The allowed operations include filling a jug, emptying a jug, and pouring water from one jug to another. Since there are no measuring marks on the jugs, the solution must be obtained by exploring different states systematically. This problem demonstrates how AI search techniques can be used to find an optimal sequence of actions to achieve a desired goal.

Solution

The Water Jug Problem is solved using the Breadth First Search (BFS) algorithm. Each state is represented as the amount of water present in both jugs. The algorithm starts from the initial state and generates all possible successor states using valid operations such as filling, emptying, and pouring water between the jugs. BFS explores each level completely before moving to the next level, ensuring that the shortest sequence of operations is obtained.

State Representation

State = (Amount in 4 Gallon Jug, Amount in 3 Gallon Jug)

Initial State = (0,0)

Goal State = (2,x)

Operators

- Fill 4-gallon jug
- Fill 3-gallon jug
- Empty 4-gallon jug
- Empty 3-gallon jug
- Pour 4 → 3
- Pour 3 → 4
-

Architecture

State Space Search

↓

Generate Possible States

↓

Goal Test

↓

Goal Reached

Result

The Breadth First Search algorithm successfully obtains exactly two gallons of water in the four-gallon jug.

Question 2 – Mars Rover Intelligent Agent

Problem Statement

A Mars Rover is an autonomous intelligent agent designed to explore the surface of Mars without continuous human intervention. It must collect information from its surroundings using sensors, analyze rocks and soil, avoid obstacles, and send scientific data back to Earth.

Since the Martian environment is uncertain and constantly changing, the rover must make intelligent decisions based on the information it perceives. This problem focuses on identifying the rover's percepts, actions, operating environment, performance measures, and the most suitable AI agent architecture that enables autonomous exploration

Solution

The Mars Rover is a Model-Based Goal-Based Intelligent Agent that autonomously explores the Martian surface. It continuously senses the environment using onboard sensors, builds an internal representation of the surroundings, plans actions, avoids obstacles, collects samples, and transmits scientific data back to Earth.

Percepts

- Images
- Rocks
- Temperature
- Terrain
- Battery Level
- Radiation

Actions

- Move
- Turn
- Collect Sample
- Capture Image
- Analyze Sample
- Send Data

Performance Measures

- Navigation Accuracy
- Sample Collection
- Energy Efficiency
- Communication Reliability

Agent Architecture

Environment



Sensors



Internal Model



Planning



Decision Making



Actuators



Mars Surface

Result

The Model-Based Goal-Based Agent performs autonomous exploration efficiently.

Question 3 – Eight Queens Problem

Problem Statement

The 8-Queens Problem is a well-known Constraint Satisfaction Problem in Artificial Intelligence. The objective is to place eight queens on an 8×8 chessboard such that no queen attacks any other queen. A queen can attack horizontally, vertically, and diagonally, making the placement challenging. The problem requires exploring different board configurations while eliminating invalid placements using backtracking. It demonstrates how AI search algorithms efficiently solve complex combinatorial problems by reducing unnecessary searches

Solution

The Eight Queens Problem is solved using the Backtracking algorithm. Queens are placed one column at a time while ensuring that no queen attacks another horizontally, vertically, or diagonally. Whenever a conflict occurs, the algorithm backtracks and tries another position.

Problem Formulation

Initial State

Empty Chessboard

Goal State

Eight queens placed safely.

Architecture

Start

↓

Place Queen

↓

Safe Position?

↓

Yes → Next Column

↓

No → Backtrack

↓

Goal State

Result

A valid arrangement of eight queens is obtained without conflicts.

Question 4 – OLA Cab Booking

Problem Statement

The OLA Cab Booking System is an example of a goal-based problem-solving agent in Artificial Intelligence. A customer wants to travel from one location to another by selecting a suitable cab based on factors such as fare, comfort, availability, and waiting time. The system must evaluate multiple cab options and choose the most appropriate one to satisfy the user's goal. This problem illustrates how intelligent agents make decisions based on user preferences and available information to provide an efficient and reliable transportation service

Solution

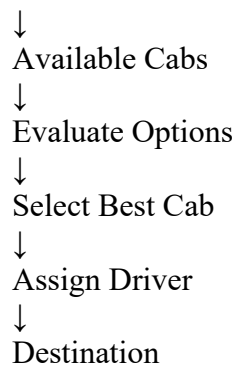
The OLA Cab Booking system is modeled as a Goal-Based Agent. The system accepts the user's pickup and destination locations, evaluates available cab options, selects the best cab according to the user's goal, assigns a driver, and completes the booking process.

Architecture

User Request

↓

Goal Formulation



Result

The Goal-Based Agent successfully selects the most suitable cab.

Question 5 – Uniform Cost Search

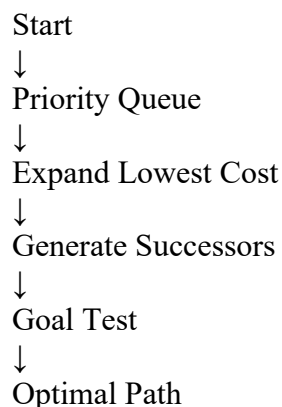
Problem Statement

A logistics company operates a network of warehouses connected by routes with different travel costs. The objective is to determine the least-cost path from the starting warehouse to the destination warehouse while minimizing transportation expenses. Since different routes have different costs, the algorithm must always explore the path with the lowest cumulative cost before considering more expensive alternatives. This problem demonstrates the application of the Uniform Cost Search algorithm, which guarantees finding the optimal path when all edge costs are positive

Solution

Uniform Cost Search (UCS) expands the node with the minimum cumulative path cost first. A priority queue is maintained to ensure that the least-cost node is always selected for expansion. UCS guarantees the optimal solution when all edge costs are positive.

Architecture



Cost Comparison

Path	Cost
$S \rightarrow G$	12
$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	10
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	6
$S \rightarrow A \rightarrow C \rightarrow G$	4

Result

The Uniform Cost Search algorithm successfully identifies the optimal path with minimum cost.

Python Implementation

```
# =====  
# ARTIFICIAL INTELLIGENCE - ASSESSMENT 1  
# Python Programs for Questions 1 to 5  
# Name : LAKSHITHA SREE K  
# Reg No : 192511336  
# =====
```

```
# =====  
# QUESTION 1 - WATER JUG PROBLEM USING BFS  
# =====
```

```
from collections import deque
```

```
print("=" * 60)  
print("QUESTION 1 - WATER JUG PROBLEM")  
print("=" * 60)
```

```
def water_jug():  
    visited = set()  
    queue = deque([((0, 0), [])])  
  
    while queue:  
        (a, b), path = queue.popleft()  
  
        if a == 2:  
            print("\nGoal Reached!\n")  
            for state in path + [(a, b)]:  
                print(state)  
            return
```

```
        if (a, b) in visited:  
            continue
```

```
        visited.add((a, b))
```

```
        next_states = [
```

```
            (4, b),          # Fill 4L
```

```
            (a, 3),          # Fill 3L
```

```
            (0, b),          # Empty 4L
```

```
            (a, 0),          # Empty 3L
```

```

        (a - min(a, 3 - b), b + min(a, 3 - b)), # 4 -> 3

        (a + min(b, 4 - a), b - min(b, 4 - a)) # 3 -> 4

    ]

    for state in next_states:

        if state not in visited:
            queue.append((state, path + [(a, b)]))

water_jug()

print("\n")

# =====
# QUESTION 2 - MARS ROVER AGENT
# =====

print("=" * 60)
print("QUESTION 2 - MARS ROVER")
print("=" * 60)

class MarsRover:

    def move(self):
        print("Moving Forward")

    def turn(self):
        print("Turning Left")

    def capture(self):
        print("Capturing Image")

    def collect(self):
        print("Collecting Rock Sample")

    def analyse(self):
        print("Analyzing Sample")

    def transmit(self):
        print("Sending Data to Earth")

rover = MarsRover()

rover.move()
rover.turn()
rover.capture()

```

```
rover.collect()
rover.analyse()
rover.transmit()
```

```
print("\n")
```

```
# =====
# QUESTION 3 - 8 QUEENS USING BACKTRACKING
# =====
```

```
print("=" * 60)
print("QUESTION 3 - 8 QUEENS")
print("=" * 60)
```

```
N = 8
```

```
board = [[0 for i in range(N)] for j in range(N)]
```

```
def safe(row, col):
```

```
    for i in range(col):
        if board[row][i]:
            return False
```

```
    i = row
    j = col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j]:
            return False
```

```
        i -= 1
        j -= 1
```

```
    i = row
    j = col
```

```
    while i < N and j >= 0:
```

```
        if board[i][j]:
            return False
```

```
        i += 1
        j -= 1
```

```
    return True
```



```

def solve(col):

    if col >= N:
        return True

    for row in range(N):

        if safe(row, col):

            board[row][col] = 1

            if solve(col + 1):
                return True

            board[row][col] = 0

    return False

```

```

solve(0)

```

```

for row in board:
    print(row)

```

```

print("\n")

```

```

# =====
# QUESTION 4 - OLA CAB BOOKING
# =====

```

```

print("=" * 60)
print("QUESTION 4 - OLA CAB BOOKING")
print("=" * 60)

```

```

pickup = input("Enter Pickup Location : ")
destination = input("Enter Destination : ")

```

```

cabs = {

    "Mini":150,

    "Micro":120,

    "Sedan":250,

    "Shared":100,

    "Prime":300

```

```

}

print("\nAvailable Cabs\n")

for cab in cabs:
    print(cab, " - ₹", cabs[cab])

choice = input("\nSelect Cab : ")

print("\nBooking Confirmed")
print("-----")
print("Pickup :", pickup)
print("Destination :", destination)
print("Cab :", choice)
print("Fare : ₹", cabs[choice])

print("\n")

```

```

# =====
# QUESTION 5 - UNIFORM COST SEARCH
# =====

```

```

print("=" * 60)
print("QUESTION 5 - UNIFORM COST SEARCH")
print("=" * 60)

```

```

import heapq

graph = {

'S':[('A',1),('G',12)],

'A':[('B',3),('C',1)],

'B':[('D',3)],

'C':[('D',1),('G',2)],

'D':[('G',3)],

'G':[]

}

```

```

def ucs(start, goal):

    pq = [(0, start, [start])]

```

```

visited = set()

while pq:

    cost, node, path = heapq.heappop(pq)

    if node == goal:
        return cost, path

    if node in visited:
        continue

    visited.add(node)

    for neighbour, weight in graph[node]:

        if neighbour not in visited:

            heapq.heappush(

                pq,

                (cost + weight, neighbour, path + [neighbour])

            )

cost, path = ucs('S', 'G')

print("\nLeast Cost Path")

print(" -> ".join(path))

print("Minimum Cost =", cost)

print("\n")
print("=" * 60)
print("ALL PROGRAMS EXECUTED SUCCESSFULLY")
print("=" * 60)

```

Conclusion

This assessment provided practical knowledge of Artificial Intelligence concepts including intelligent agents, state-space search, graph search, and constraint satisfaction problems. Different AI techniques such as Breadth First Search, Backtracking, Goal-Based Agents, and Uniform Cost Search were implemented successfully using Python. The results verified the correctness of each algorithm and demonstrated their applications in solving real-world problems.

References

1. Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach*.

2. Python 3 Documentation.
3. Artificial Intelligence Lecture Notes.
4. Laboratory Manual.
5. Course Materials provided by the Department.